

SWEET CUPCAKE SHOP MANAGEMENT SYSTEM

SYSTEM DOCUMENTATION REPORT

Student Name: Malith Pabasara

Student ID: CL/HDCSE/CMU/132/52

Module: CSE4006 - Object Oriented Programming

Module Leader: upeka@icbtcampus.edu.lk

Institution: Cardiff Metropolitan University (ICBT Campus)

Submission Date: January 27, 2026



Cardiff
Metropolitan
University

Prifysgol
Metropolitan
Caerdydd

TABLE OF CONTENT

1. EXECUTIVE SUMMARY	3
2. SYSTEM OVERVIEW.....	4
2.1 Purpose.....	4
2.2 Scope.....	4
2.3 System Requirements	4
3. UML DIAGRAMS	5
3.1 Use Case Diagram	5
3.2 Class Diagram	8
3.3 Sequence Diagrams	14
4. SYSTEM DEVELOPMENT & OOP CONCEPTS	19
4.1 Object-Oriented Programming Concepts.....	19
4.2 System Architecture	34
4.3 File Handling Implementation	36
4.4 Code Documentation	42
5. USER MANUAL	46
6. SYSTEM DEMONSTRATION	46
7. CONCLUSION	48
8. REFERENCES.....	52

1. EXECUTIVE SUMMARY

The Sweet Cupcake Shop Management System is a Desktop Application developed using Java and Object – Oriented Programming principles to streamline cupcake inventory management for bakery staff. The system implements role-based access control with two distinct user roles: Cashier and Manager, each with Specific permissions and functionalities

The application was developed using Java Swing for the graphical user interface and implements file-based data persistence for storing cupcake inventory and user account information. The system demonstrates all six core Object-Oriented Programming concepts: Objects, Classes, Abstraction, Inheritance, Encapsulation, and Polymorphism.

Key Features

- Secure user authentication with role-based access
- Cupcake inventory management (add, view, search)
- Category-based organization (6 categories)
- User account management (Manager only)
- File-based data persistence
- Professional GUI with intuitive navigation

Technologies Used

- Programming Language: Java (JDK 8+)
- GUI Framework: Java Swing
- Development Environment: NetBeans IDE
- Data Storage: Text file-based persistence

2. SYSTEM OVERVIEW

2.1 Purpose

The Sweet Cupcake Shop Management System was designed to address the need for an efficient, user-friendly inventory management solution for small to medium-sized bakery businesses. The system automates manual record-keeping processes and provides staff with quick access to product information.

2.2 Scope

The system focuses on:

- **Staff Management:** Role-based access for Cashiers and Managers
- **Inventory Management:** Complete cupcake product lifecycle management
- **Data Persistence:** Reliable file-based storage system
- **User Experience:** Intuitive interface requiring minimal training

2.3 System Requirements

Hardware:

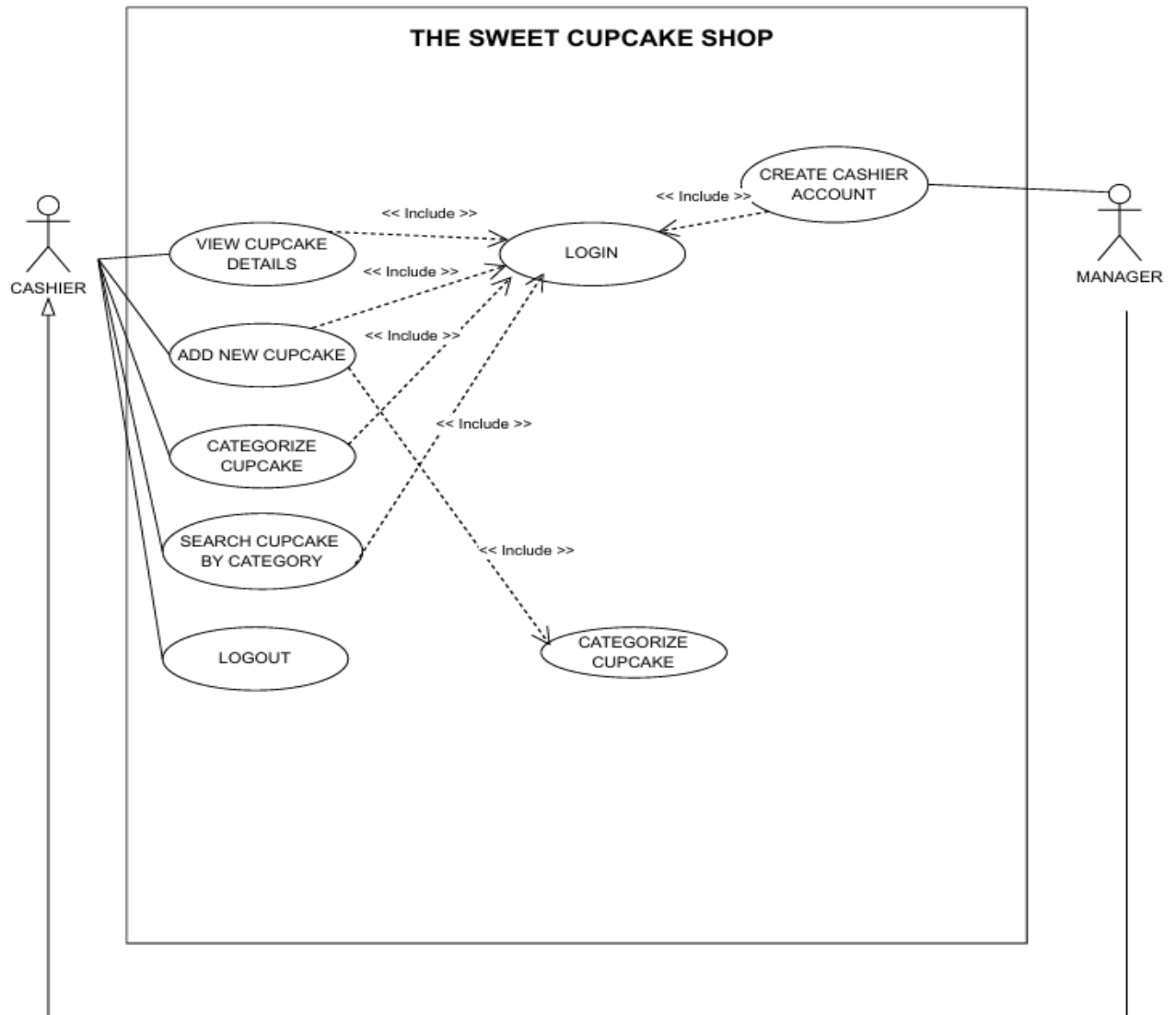
- Processor: Intel Core i3 or equivalent
- RAM: 4GB minimum
- Storage: 100MB free space
- Display: 1024x768 resolution minimum

Software:

- Operating System: Windows 7+, macOS, or Linux
- Java Runtime Environment (JRE) 8 or higher

3. UML DIAGRAMS

3.1 Use Case Diagram



Purpose: The Use Case Diagram illustrates the functional requirements of the system and shows how different actors interact with the system.

Actors:

1. Cashier - Front-line staff member responsible for daily inventory operations
2. Manager - Supervisor with full system access including user management

Use Case:

1. Login

- **Description:** Authenticates user credentials and grants role-based access
- **Actors:** Both Cashier and Manager
- **Precondition:** User must have valid account
- **Postcondition:** User accesses appropriate dashboard

2. View Cupcake Details

- **Description:** Displays complete inventory of all cupcakes
- **Actors:** Both Cashier and Manager
- **Include:** Login (must be authenticated)

3. Add New Cupcake

- **Description:** Creates new cupcake entry in inventory
- **Actors:** Both Cashier and Manager
- **Include:** Login, Categorize Cupcake

4. Categorize Cupcake

- **Description:** Assigns category to cupcake product
- **Categories:** CLASSIC_FLAVORS, RED_VELVET, FRUITY_DELIGHTS, SEASONAL_SPECIALS, CUSTOM_ORDERS, GLUTEN_FREE

5. Search Cupcake by Category

- **Description:** Filters inventory by selected category
- **Actors:** Both Cashier and Manager
- **Include:** Login

6. Create Cashier Account

- **Description:** Creates new Cashier user account
- **Actors:** Manager only
- **Include:** Login
- **Business Rule:** Only Managers can create Cashier accounts

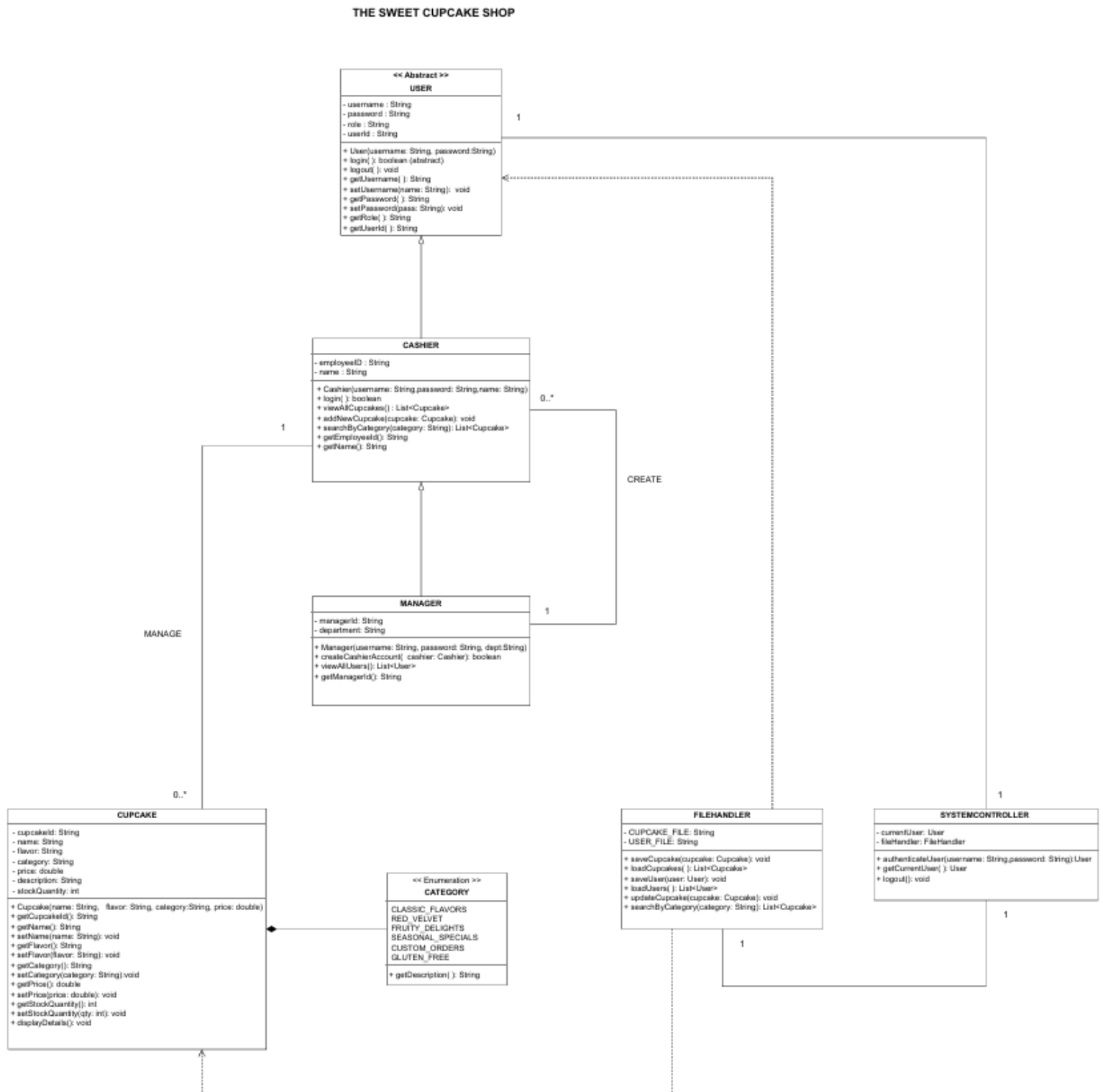
7. Logout

- **Description:** Terminates session and returns to login screen
- **Actors:** Both Cashier and Manager

Relationships:

- **Generalization:** Manager —▷ Cashier
Manager inherits all Cashier capabilities plus additional privileges
- **Include:** All use cases <<include>> Login
Authentication required before any system operation
- **Include:** Add New Cupcake <<include>> Categorize Cupcake
Categorization is mandatory when adding cupcakes

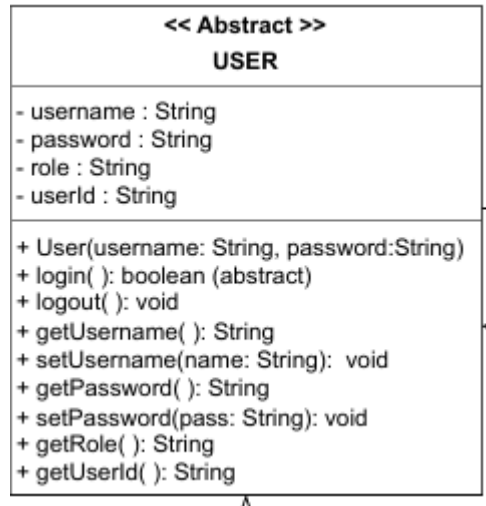
3.2 Class Diagram



Purpose: The Class Diagram represents the static structure of the system, showing all classes, their attributes, methods, and relationships.

Classes:

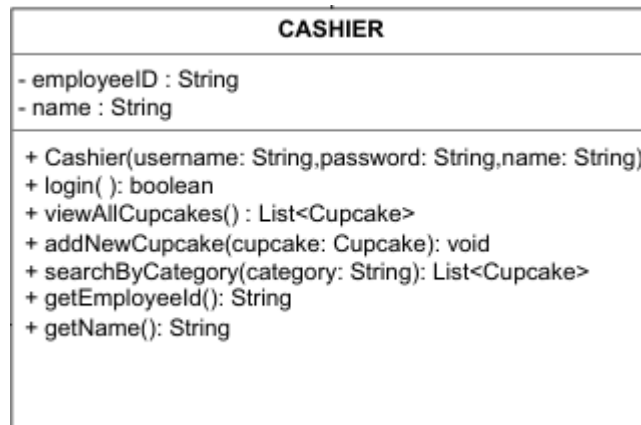
1.User (Abstract)



Explanation:

- **Abstract class** demonstrating **Abstraction** concept
- Cannot be instantiated directly
- Provides common attributes for all user types
- Abstract **login ()** method must be implemented by child classes
- Demonstrates **Encapsulation** with private attributes and public getters/setters

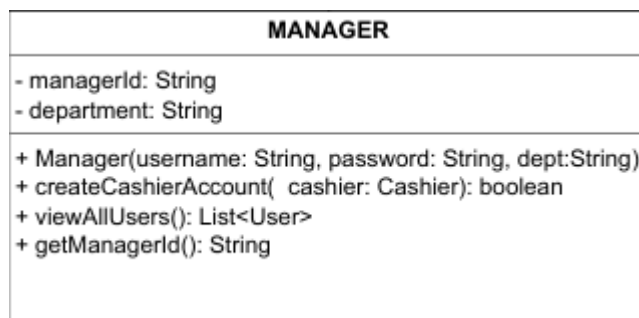
2. Cashier (Extends User)



Explanation:

- **Inherits** from User class demonstrating **Inheritance**
- Implements abstract **login ()** method showing **Polymorphism**
- Contains Cashier-specific attributes and methods
- Demonstrates **Method Overriding**

3. Manager (Extends Cashier)



Explanation:

- **Multi-level inheritance:** Manager → Cashier → User
- Inherits ALL Cashier capabilities
- Adds Manager-specific functionalities
- Demonstrates **"IS-A" relationship:** Manager IS-A Cashier IS-A User

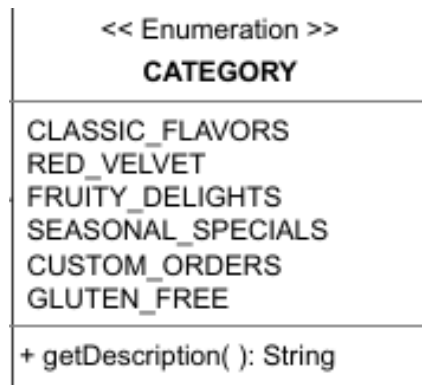
4. Cupcake

CUPCAKE
- cupcakeld: String - name: String - flavor: String - category: String - price: double - description: String - stockQuantity: int
+ Cupcake(name: String, flavor: String, category:String, price: double) + getCupcakeld(): String + getName(): String + setName(name: String): void + getFlavor(): String + setFlavor(flavor: String): void + getCategory(): String + setCategory(category: String):void + getPrice(): double + setPrice(price: double): void + getStockQuantity(): int + setStockQuantity(qty: int): void + displayDetails(): void

Explanation:

- Represents cupcake product entity
- **Constructor overloading** (two constructors)
- Complete **Encapsulation** with private attributes
- All data access through public methods

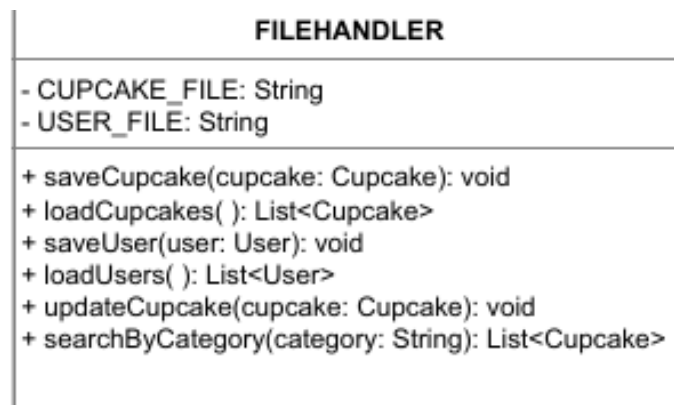
5. Category (Enumeration)



Explanation:

- Defines fixed set of cupcake categories
- Type-safe constant values
- Prevents invalid category assignments

6. File Handler



Explanation:

- Handles all file I/O operations
- Implements data persistence
- Separates data access logic from business logic

7. System Controller

SYSTEMCONTROLLER
- currentUser: User - fileHandler: FileHandler
+ authenticateUser(username: String,password: String):User + getCurrentUser(): User + logout(): void

Explanation:

- Controls system flow and authentication
- Manages user sessions
- Coordinates between UI and data layers

Class Relationship

1. Inheritance (Generalization):

- Cashier —▷ User (hollow triangle arrow)
- Manager —▷ Cashier (hollow triangle arrow)

Demonstrates IS-A relationship

2. Association:

- Cashier — Cupcake [1 manages 0..*]
- Manager — Cashier [1 creates 0..*]
- SystemController — User [1 to 1]
- SystemController — FileHandler [1 to 1]

3. Composition:

- Cupcake ◆———— Category (filled diamond)

Category cannot exist without Cupcake

4. Dependency:

- FileHandler> Cupcake (dashed arrow)
- FileHandler> User (dashed arrow)

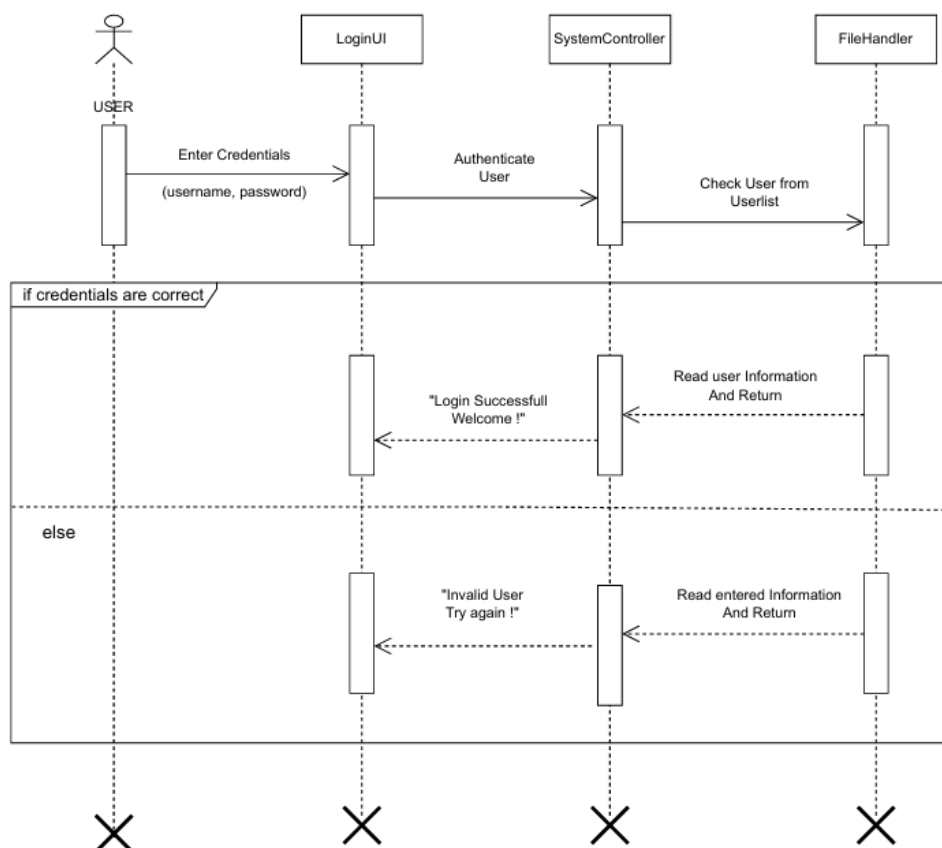
FileHandler depends on these classes

3.3 Sequence Diagrams

Sequence Diagrams show how objects interact in a particular scenario by illustrating the messages passed between objects over time

1. User Login

1. USER LOGIN



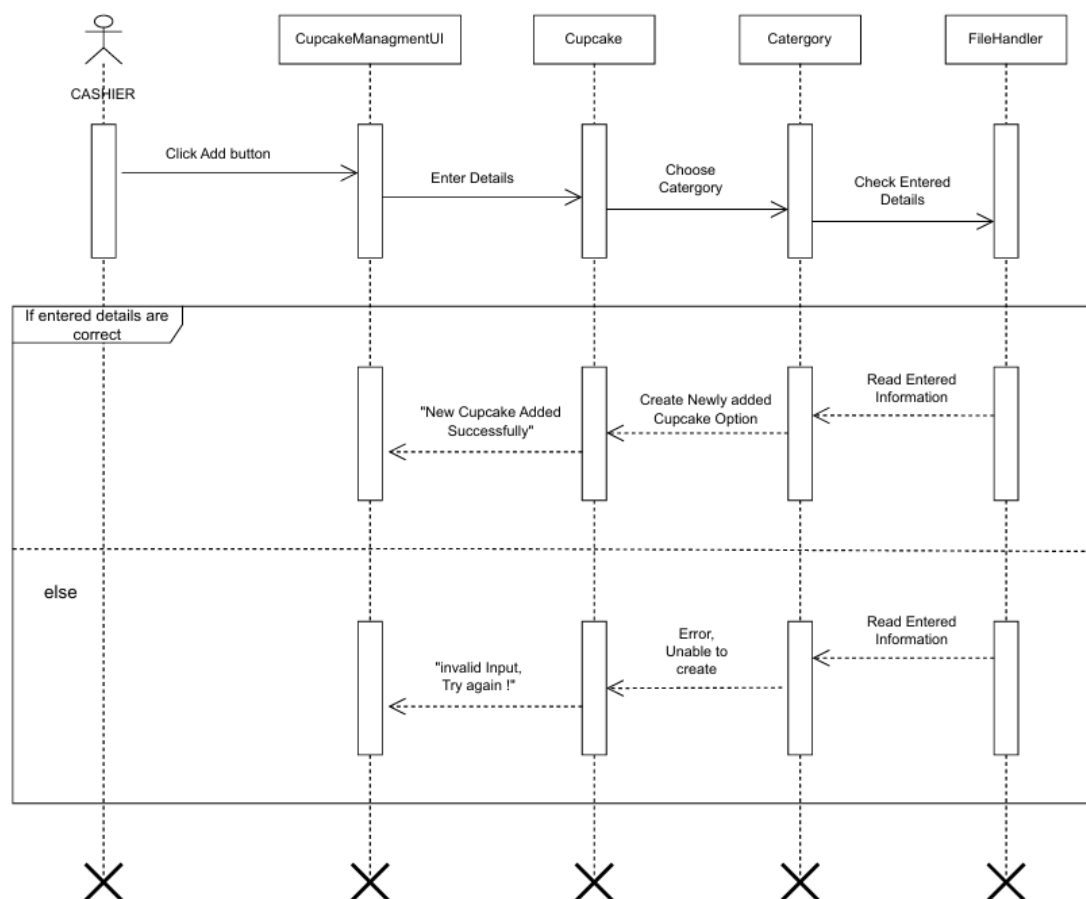
Purpose: Demonstrates the authentication process when a user logs into the system.

Participants:

- **User:** The person attempting to login (actor)
- **LoginUI:** Login interface window
- **SystemController:** Manages authentication logic
- **FileHandler:** Loads user data from file

2. Add new Cupcake

2. ADD NEW CUPCAKE



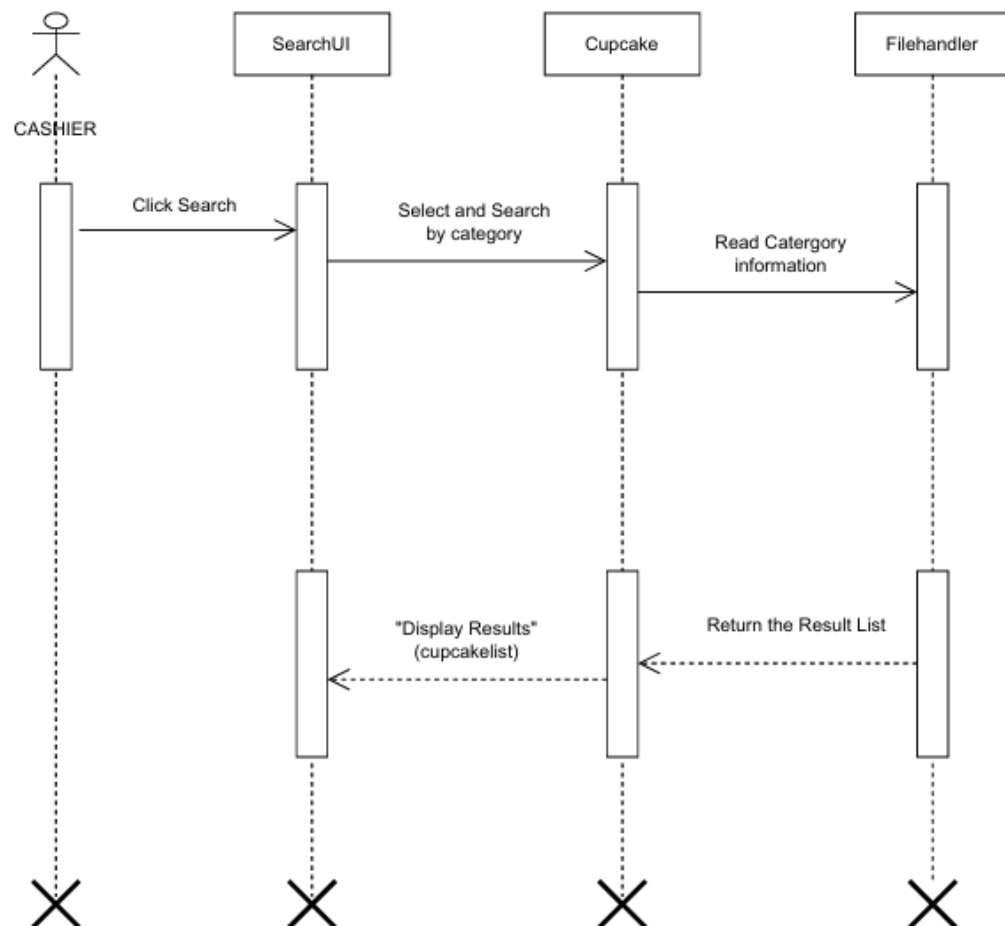
Purpose: Shows the process of adding a new cupcake to the inventory.

Participants:

- **Cashier:** User performing the action (actor)
- **CupcakeManagementUI:** Add cupcake dialog window
- **Cupcake:** New cupcake object to be created
- **Category:** Enumeration for cupcake categories
- **FileHandler:** Saves cupcake data to file

3. Search Cupcake by Category

3. SEARCH CUPCAKE BY CATERGORY



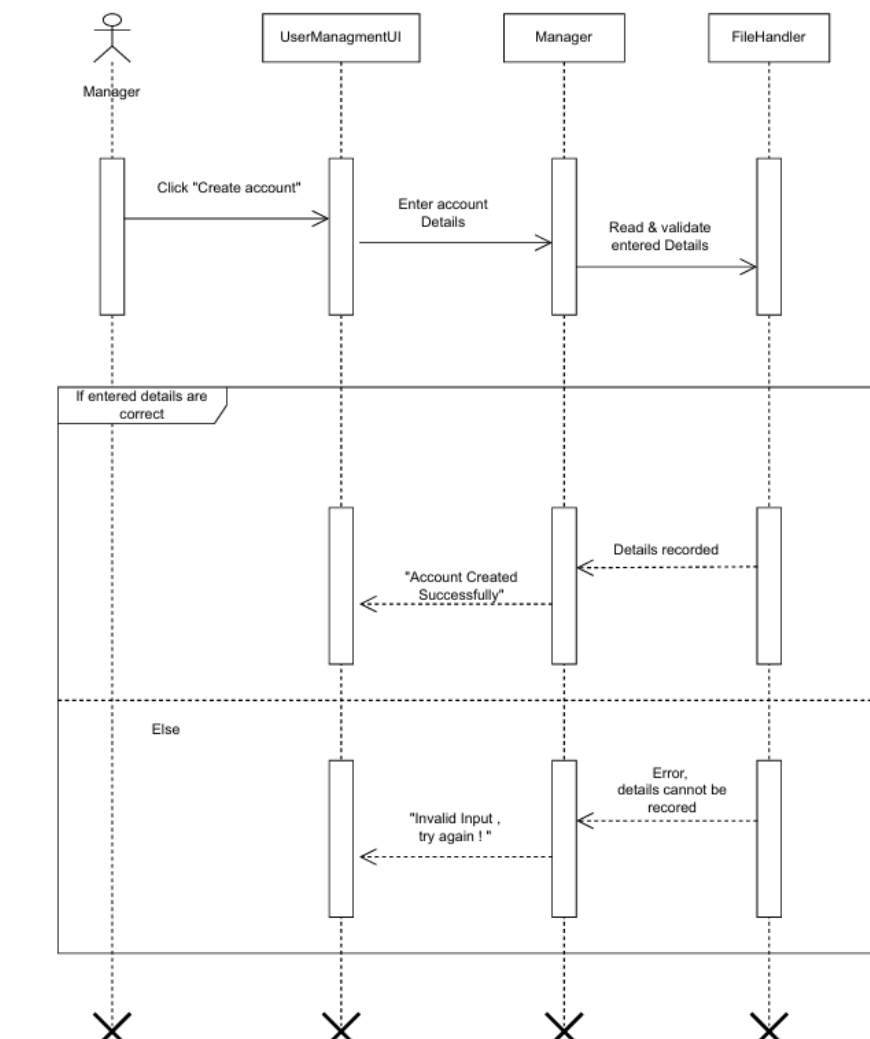
Purpose: Illustrates how the system searches and filters cupcakes by category.

Participants:

- **Cashier:** User performing search (actor)
- **SearchUI:** Search dialog window
- **FileHandler:** Loads and filters cupcake data
- **Cupcake(s):** Multiple cupcake objects in file

4. Create Cashier Account

4. CREATE CASHIER ACCOUNT (MANAGER ONLY)



Purpose: Shows Manager-specific functionality for creating new Cashier accounts.

Participants:

- **Manager:** Supervisor creating account (actor)
- **UserManagementUI:** Create account dialog
- **Manager Object:** Logged-in Manager instance
- **FileHandler:** Saves new user to file

Summary of Sequence Diagrams:

All four sequence diagrams demonstrate:

- **Object interaction** through method calls
- **Temporal ordering** of events (top to bottom)
- **Conditional logic** using alt frames
- **Iteration** using loop frames
- **Return values** shown with dashed arrows
- **Activation bars** showing when objects are processing
- **Lifelines** representing object existence over time

4. SYSTEM DEVELOPMENT & OOP CONCEPTS

This Section documents the implementation of the Sweet Cupcake Shop Management System and demonstrates how all six Object-Oriented Programming concepts have been applied throughout the codebase.

4.1 Object-Oriented Programming Concepts

Object Oriented Programming (OOP) is a programming paradigm based on the concept of “Objects” which contain data and code. The Sweet Cupcake Shop System implements all Six OOP Concepts as required by the assignment

1. Object

Definition: An object is an instance of a class that contains state (attributes) and behavior (methods).

Objects are created throughout the system to represent real-world entities

Ex1. Creating Cupcake Objects

```
Cupcake chocolateCupcake = new Cupcake(  
    "Chocolate Delight",           // name  
    "Rich Chocolate",             // flavor  
    "CLASSIC_FLAVORS",           // category  
    3.50,                         // price  
    "Decadent chocolate cupcake", // description  
    50                            // stock quantity  
);
```

Ex2. Creating User Objects

```
Cashier cashier = new Cashier("john_doe", "password123", "John Doe");
```

```
Manager manager = new Manager("admin", "admin123", "Admin Manager",  
"Management");
```

Real – World Mapping

- Each Cupcake object represents an actual cupcake product in inventory
- Each Cashier object represents a real cashier employee
- Each Manager object represents a real manager

Benefits:

- Objects encapsulate related data and behavior
- Easy to create multiple instances
- Represents real-world entities naturally

2. Class

Definition: A class is a blueprint or template for creating objects. It defines the structure and behavior that objects will have.

The system contains 7 main classes:

1. **User** (abstract class)
2. **Cashier**
3. **Manager**
4. **Cupcake**
5. **Category** (enumeration)
6. **FileHandler**
7. **SystemController**

Ex1. Cupcake Class Structure

```
public class Cupcake implements Serializable {  
    // Attributes (State)  
    private String cupcakeId;  
    private String name;  
    private String flavor;  
    private String category;  
    private double price;  
    private String description;  
    private int stockQuantity;  
  
    // Constructor (Object Creation)  
    public Cupcake(String name, String flavor, String category, double price) {  
        this.cupcakeId = "CUP" + System.currentTimeMillis();  
        this.name = name;  
        this.flavor = flavor;  
        this.category = category;  
        this.price = price;  
        this.description = "";  
        this.stockQuantity = 0;  
    }  
  
    // Methods (Behavior)  
    public void displayDetails() {  
        System.out.println("ID: " + cupcakeId);  
        System.out.println("Name: " + name);  
        System.out.println("Price: $" + price);  
    }  
}
```

```
// Getters and Setters

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}
}
```

Class Components:

- **Attributes:** Define what data the class holds
- **Constructor:** Initializes objects when created
- **Methods:** Define what actions objects can perform
- **Access Modifiers:** Control visibility (private, public)

3. Abstraction

Definition: Abstraction is hiding complex implementation details and showing only essential features. It is achieved through abstract classes and interfaces.

Ex1. Abstract User Class

```
public abstract class User implements Serializable {  
  
    // Private attributes  
  
    private String username;  
  
    private String password;  
  
    private String role;  
  
    private String userId;  
  
  
    // Constructor  
  
    public User(String username, String password, String role) {  
        this.username = username;  
        this.password = password;  
        this.role = role;  
        this.userId = generateUserId();  
    }  
  
  
    // ABSTRACT METHOD - Must be implemented by child classes  
    public abstract boolean login(String username, String password);  
  
  
    // CONCRETE METHOD - Shared by all users  
    public void logout() {  
        System.out.println(username + " logged out successfully.");  
    }  
}
```

```
// Getters and Setters

public String getUsername() {

    return username;

}

}
```

Key Points:

- User class is **abstract** - cannot be instantiated directly
- **login()** method is **abstract** - no implementation in User class
- Child classes **must** implement **login()** method
- **logout()** is concrete - shared by all users

Why Abstraction?

- User is a general concept - real users are either Cashiers or Managers
- Different user types login differently (different validation rules)
- Enforces consistency - all user types must implement login
- Hides complexity - GUI doesn't need to know login details

Cannot Do This

```
User user = new User("test", "pass", "role"); // ERROR! Cannot instantiate abstract class
```

Only Accessible by

```
User cashier = new Cashier("test", "pass", "John"); // OK! Cashier is concrete
```

```
User manager = new Manager("admin", "pass", "Admin", "IT"); // OK! Manager is concrete
```


4. Inheritance

Definition: Inheritance allows a class to inherit properties and methods from another class. It establishes an "IS-A" relationship between classes.

The system demonstrates “**multi-level inheritance**”

Ex1. Cashier Inherits from User

```
public class Cashier extends User {  
    // Additional Cashier-specific attributes  
    private String employeeId;  
    private String name;  
  
    // Constructor calls parent constructor  
    public Cashier(String username, String password, String name) {  
        super(username, password, "Cashier"); // Call User constructor  
        this.name = name;  
        this.employeeId = "EMP" + System.currentTimeMillis();  
    }  
  
    // Implement abstract method from User  
    @Override  
    public boolean login(String username, String password) {  
        return this.getUsername().equals(username) &&  
            this.getPassword().equals(password);  
    }  
}
```

```

// Cashier-specific methods

public void addNewCupcake(Cupcake cupcake, List<Cupcake> cupcakeList) {
    cupcakeList.add(cupcake);
    System.out.println("Cupcake added: " + cupcake.getName());
}
}

```

Ex2. Manager Inherits from Cashier

```

public class Manager extends Cashier {
    // Additional Manager-specific attributes

    private String managerId;
    private String department;

    // Constructor calls parent constructor

    public Manager(String username, String password, String name, String department)
    {
        super(username, password, name); // Call Cashier constructor
        this.department = department;
        this.managerId = "MGR" + System.currentTimeMillis();
    }

    // Manager-specific method

    public boolean createCashierAccount(Cashier cashier, List<User> userList) {
        if (cashier != null) {
            userList.add(cashier);
            System.out.println("Cashier created: " + cashier.getUsername());
            return true;
        }
        return false;
    }
}

```

Inheritance Hierarchy

Manager inherits:

- From Cashier: employeeId, name, **addNewCupcake()**, **searchByCategory()**
- From User: username, password, role, userId, **logout()**

IS-A Relationships:

- Manager IS-A Cashier
- Manager IS-A User
- Cashier IS-A User

Benefits:

- **Code Reusability:** Manager gets all Cashier methods without rewriting
- **Logical Hierarchy:** Reflects real-world relationships
- **Maintainability:** Changes to User automatically apply to Cashier and Manager

5. Encapsulation

Definition: Encapsulation is the bundling of data (attributes) and methods that operate on that data within a single unit (class), and restricting direct access to some of the object's components. This is achieved using access modifiers (private, public, protected)

Ex1. Cupcake Class Encapsulation

```
public class Cupcake implements Serializable {  
  
    // PRIVATE attributes - Cannot be accessed directly from outside  
  
    private String cupcakeId;  
  
    private String name;  
  
    private String flavor;  
  
    private String category;  
  
    private double price;  
  
    private String description;  
  
    private int stockQuantity;  
  
  
    // PUBLIC constructor - Can be called from outside  
  
    public Cupcake(String name, String flavor, String category, double price) {  
  
        this.cupcakeId = "CUP" + System.currentTimeMillis();  
  
        this.name = name;  
  
        this.flavor = flavor;  
  
        this.category = category;  
  
        this.price = price;  
  
    }  
}
```

```
// PUBLIC getter - Controlled read access

public String getName() {

    return name;

}

// PUBLIC setter - Controlled write access with validation

public void setPrice(double price) {

    if (price > 0) { // Validation

        this.price = price;

    } else {

        System.out.println("Price must be positive!");

    }

}

// PUBLIC method - Controlled behavior

public void displayDetails() {

    System.out.println("Name: " + name);

    System.out.println("Price: $" + price);

}

}
```

Why Encapsulation?

Without Encapsulation anyone can modify and set invalid values

```
public class Cupcake {  
    public String name; // Public - anyone can modify  
    public double price; // Public - anyone can set invalid values  
}
```

// Problem: No control over data

```
Cupcake cupcake = new Cupcake();  
cupcake.price = -5.00; // Invalid! But no validation  
cupcake.name = "";    // Invalid! But no check
```

Benefits:

- **Data Protection:** Private attributes cannot be modified directly
- **Validation:** Setters can validate data before assignment
- **Flexibility:** Implementation can change without affecting external code
- **Control:** Class controls how its data is accessed and modified

Ex2. User Class Encapsulation part

```
public abstract class User {  
    private String username; // Cannot be changed after creation  
    private String password; // Protected from direct access  
  
    // Getter - Read access only  
    public String getUsername() {  
        return username;  
    }  
  
    // No setUsername() - Username is immutable after creation  
    // Password can only be changed through controlled method  
}
```

6. Polymorphism

Definition: Polymorphism means "many forms." It allows objects of different classes to be treated as objects of a common parent class. The same method name can behave differently in different classes.

Types of Polymorphism:

1. **Method Overriding** (Runtime Polymorphism)
2. **Method Overloading** (Compile-time Polymorphism)

Method Overriding in User (Abstract)

```
public abstract boolean login(String username, String password);
```

Method Overriding in Cashier

@Override

```
public boolean login(String username, String password) {  
    // Cashier-specific login logic  
    return this.getUsername().equals(username) &&  
        this.getPassword().equals(password);  
}
```


Method Overloading

Multiple Constructors in Cupcake Class

```
public class Cupcake {  
    // Constructor 1 - Basic cupcake  
    public Cupcake(String name, String flavor, String category, double price) {  
        this.cupcakeId = "CUP" + System.currentTimeMillis();  
        this.name = name;  
        this.flavor = flavor;  
        this.category = category;  
        this.price = price;  
        this.description = "";  
        this.stockQuantity = 0;  
    }  
  
    // Constructor 2 - Complete cupcake with all details  
    public Cupcake(String name, String flavor, String category, double price,  
        String description, int stockQuantity) {  
        this(name, flavor, category, price); // Call first constructor  
        this.description = description;  
        this.stockQuantity = stockQuantity;  
    }  
}
```

Benefits of Polymorphism:

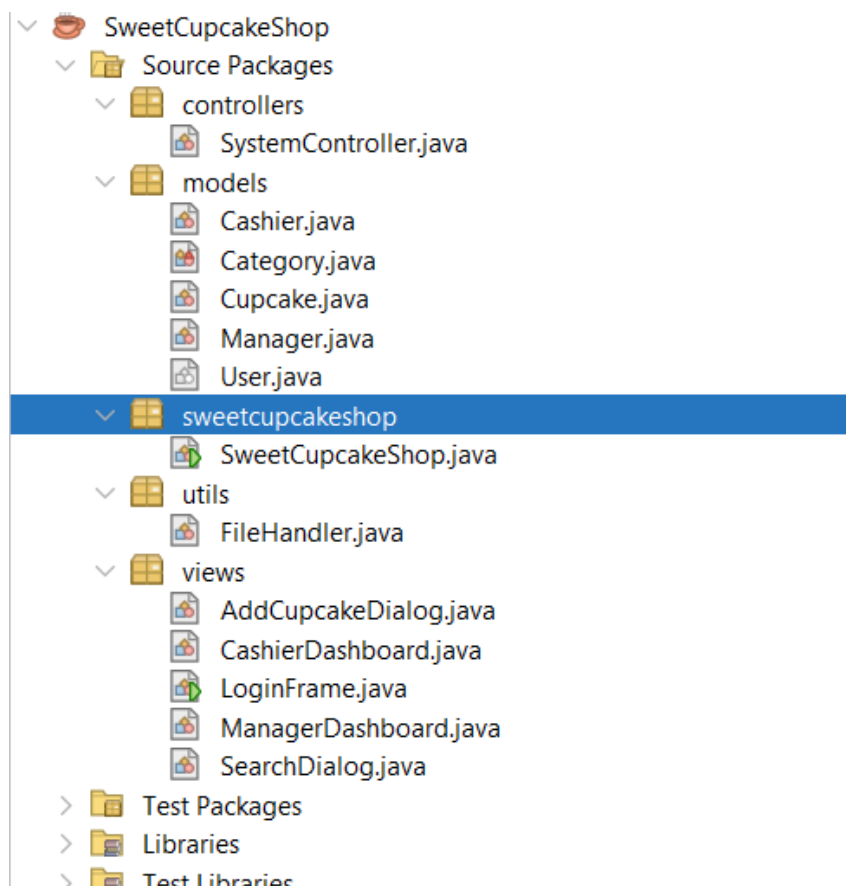
- **Flexibility:** Same interface, different implementations
- **Code Reusability:** Parent class reference can hold child objects
- **Extensibility:** New classes can be added without changing existing code
- **Dynamic Behavior:** Method called is determined at runtime

Summary of OOP Concepts

Concept	Implementation	Benefit
Object	Cupcake, Cashier, Manager instances	Represents real-world entities
Class	User, Cupcake, FileHandler	Blueprint for objects
Abstraction	Abstract User class, abstract login()	Hides complexity, enforces structure
Inheritance	Manager→Cashier→User	Code reusability, logical hierarchy
Encapsulation	Private attributes, public getters/setters	Data protection, controlled access
Polymorphism	Overridden login(), Constructor overloading	Flexibility, dynamic behavior

4.2 System Architecture

The Sweet Cupcake Shop system follows a layered architecture with clear separation of concerns, organized into logical packages



Architectural Layers

1. Presentation Layer (Views)

- **Responsibility:** User interface and user interaction
- **Components:** All GUI classes (JFrame, JDialog)
- **Communication:** Interacts with Controller layer only
- **Technology:** Java Swing

2. Business Logic Layer (Controllers)

- **Responsibility:** Application logic, authentication, session management
- **Components:** SystemController
- **Communication:** Bridges Views and Data layers
- **Functions:** Validates input, enforces business rules

3. Data Layer (Models)

- **Responsibility:** Data structure and domain objects
- **Components:** User, Cashier, Manager, Cupcake, Category
- **Communication:** Used by all layers
- **Characteristics:** POJOs (Plain Old Java Objects)

4. Utility Layer (Utils)

- **Responsibility:** File operations and helper functions
- **Components:** FileHandler
- **Communication:** Called by Controller and View layers
- **Functions:** Read/write files, data persistence

Benefits of This Architecture:

- **Separation of Concerns:** Each layer has distinct responsibility
- **Maintainability:** Changes to UI don't affect data layer
- **Testability:** Layers can be tested independently
- **Scalability:** Easy to add new features or change components
- **Reusability:** Models and utilities can be reused

4.3 File Handling Implementation

The system uses text file-based persistence for data storage, providing a simple yet effective solution for small to medium-scale applications

Files:

1. **cupcakes.txt** - Stores all cupcake inventory data
2. **users.txt** - Stores all user account information

Location: `/data/` directory (auto-created on first launch)

Ex. cupcakes.txt Format

```
CUP1738023456789|Chocolate Delight|Rich Chocolate|CLASSIC  
FLAVORS|3.50|Decadent chocolate cupcake|50
```

```
CUP1738023456790|Vanilla Dream|Vanilla Bean|CLASSIC FLAVORS|3.00|Classic  
vanilla cupcake|40
```

```
CUP1738023456791|Red Velvet Supreme|Red Velvet|RED VELVET|4.00|Smooth red  
velvet|30
```

Format: id|name|flavor|category|price|description|stock

- Fields separated by pipe character (|)
- One cupcake per line
- All fields required

FileHandler Operations

1. Save Cupcake

```
public void saveCupcake(Cupcake cupcake) {  
  
    try (BufferedWriter writer = new BufferedWriter(  
        new FileWriter(CUPCAKE_FILE, true))) {  
  
        writer.write(cupcakeToString(cupcake));  
  
        writer.newLine();  
  
        System.out.println("Cupcake saved successfully!");  
  
    } catch (IOException e) {  
  
        System.err.println("Error saving cupcake: " + e.getMessage());  
  
    }  
}
```

2. Load Cupcakes

```
public List<Cupcake> loadCupcakes() {  
    List<Cupcake> cupcakes = new ArrayList<>();  
  
    try (BufferedReader reader = new BufferedReader(  
        new FileReader(CUPCAKE_FILE))) {  
  
        String line;  
        while ((line = reader.readLine()) != null) {  
            if (!line.trim().isEmpty()) {  
                Cupcake cupcake = stringToCupcake(line);  
                if (cupcake != null) {  
                    cupcakes.add(cupcake);  
                }  
            }  
        }  
  
    } catch (IOException e) {  
        System.err.println("Error loading cupcakes: " + e.getMessage());  
    }  
  
    return cupcakes;  
}
```

3. Search by Category

```
public List<Cupcake> searchByCategory(String category) {  
    List<Cupcake> allCupcakes = loadCupcakes();  
    List<Cupcake> results = new ArrayList<>();  
  
    for (Cupcake cupcake : allCupcakes) {  
        if (cupcake.getCategory().equalsIgnoreCase(category)) {  
            results.add(cupcake);  
        }  
    }  
  
    return results;  
}
```

Data Conversion Methods

- Converting Object to String (Serialization)

```
private String cupcakeToString(Cupcake cupcake) {  
    return cupcake.getCupcakeId() + "|" +  
        cupcake.getName() + "|" +  
        cupcake.getFlavor() + "|" +  
        cupcake.getCategory() + "|" +  
        cupcake.getPrice() + "|" +  
        cupcake.getDescription() + "|" +  
        cupcake.getStockQuantity();  
}
```

Converting String to Object (Deserialization)

```
private Cupcake stringToCupcake(String line) {  
    try {  
        String[] parts = line.split("\\|");  
        if (parts.length >= 7) {  
            Cupcake cupcake = new Cupcake(  
                parts[1],           // name  
                parts[2],           // flavor  
                parts[3],           // category  
                Double.parseDouble(parts[4]), // price  
                parts[5],           // description  
                Integer.parseInt(parts[6])    // stock  
            );  
            return cupcake;  
        }  
    } catch (Exception e) {  
        System.err.println("Error parsing cupcake: " + e.getMessage());  
    }  
    return null;  
}
```


Error Handling

Try-Catch Blocks:

- All file operations wrapped in try-catch
- Prevents crashes from file access errors
- Logs errors to console
- Returns empty lists on failure

File Existence Checks

```
File file = new File(CUPCAKE_FILE);  
  
if (!file.exists()) {  
  
    System.out.println("File not found. Creating new file...");  
  
    return new ArrayList<>();  
  
}
```

Benefits of File-Based Storage

- **Simplicity:** No database setup required
- **Portability:** Files travel with application
- **Human-Readable:** Easy to debug
- **Lightweight:** No external dependencies
- **Sufficient:** Adequate for small-scale systems

Limitations:

- Not suitable for large datasets (1000+ records)
- No concurrent access control
- No query optimization
- Manual data integrity management

4.4 Code Documentation

This section provides an overview of key classes and their implementation details

The Main entry Point

```
public class SweetCupcakeShop {  
  
    public static void main(String[] args) {  
  
        // Initialize system  
  
        initializeSystem();  
  
  
        // Set Look and Feel  
  
        setLookAndFeel();  
  
  
        // Launch application  
  
        SwingUtilities.invokeLater(new Runnable() {  
  
            @Override  
  
            public void run() {  
  
                LoginFrame loginFrame = new LoginFrame();  
  
                loginFrame.setVisible(true);  
  
            }  
  
        });  
  
    }  
}
```

Responsibilities:

- Initialize data directory
- Create sample data if needed
- Set UI theme
- Launch login screen

Key Overview of Classes

Category.java (Enumeration)

- Defines 6 cupcake categories
- Type-safe constants
- Includes description for each category
- **Lines of Code:** 30

User.java (Abstract Class)

- Base class for all users
- Common attributes: username, password, role, userId
- Abstract login() method
- Concrete logout() method
- **Lines of Code:** 70

Cashier.java

- Extends User
- Implements Cashier-specific operations
- Methods: viewAllCupcakes(), addNewCupcake(), searchByCategory()
- **Lines of Code:** 90

Manager.java

- Extends Cashier (multi-level inheritance)
- Adds Manager-specific operations
- Methods: createCashierAccount(), viewAllUsers()
- **Lines of Code:** 80

Cupcake.java

- Represents cupcake entity
- Complete encapsulation
- Constructor overloading
- **Lines of Code:** 120

FileHandler.java

- Handles all file I/O
- CRUD operations for cupcakes and users
- Error handling for file operations
- **Lines of Code:** 250

SystemController.java

- Manages authentication
- Session management
- Bridges UI and data layers
- **Lines of Code:** 150

LoginFrame.java

- Login user interface
- Credential validation
- Role-based navigation
- **Lines of Code:** 180

CashierDashboard.java

- Main Cashier interface
- Cupcake management features
- Table display with data
- **Lines of Code: 220**

ManagerDashboard.java

- Extends CashierDashboard
- Adds Manager-only features
- User account management
- **Lines of Code: 180**

AddCupcakeDialog.java

- Form for adding cupcakes
- Input validation
- Category dropdown
- **Lines of Code: 200**

SearchDialog.java

- Search interface
- Category filtering
- Results display
- **Lines of Code: 150**

Total Lines of Code: 1,720

5. USER MANUAL

A comprehensive User Manual has been created as a separate document covering:

- System installation and requirements
- Login procedures
- Cashier functionalities
- Manager functionalities
- Troubleshooting guide

[SWEET CUPCAKE SHOP SYSTEM.pdf](#)

6. SYSTEM DEMONSTRATION

The system demonstration will showcase

1. Login Functionality:

- Successful login as Cashier
- Successful login as Manager
- Failed login with invalid credentials

2. Cashier Features:

- View all cupcakes in inventory
- Add new cupcake with all details
- Search cupcakes by category
- Logout functionality

3. Manager Features:

- All Cashier features (inheritance demonstration)
- Create new Cashier account
- View all system users
- Validation of duplicate usernames

4. Data Persistence:

- Changes persist after application restart
- File-based storage demonstration
- Data integrity verification

5. Error Handling:

- Invalid input validation
- Empty field checks
- Price and stock validation
- User-friendly error messages

6. OOP Concepts Demonstration:

- **Abstraction:** Cannot create User object directly
- **Inheritance:** Manager has all Cashier capabilities
- **Encapsulation:** Cannot directly modify private attributes
- **Polymorphism:** Different login behavior for different users

7. CONCLUSION

The Sweet Cupcake Shop Management System successfully demonstrates the practical application of Object-Oriented Programming principles in developing a real-world business application. The system provides a robust, user-friendly solution for managing cupcake inventory and user accounts in a bakery setting.

Learning Outcomes

Through this project, the following learning outcomes were achieved:

1. Object-Oriented Design:

- Understanding of class hierarchies
- Effective use of inheritance and polymorphism
- Application of abstraction principles
- Implementation of encapsulation

2. Software Development:

- Requirements analysis and design
- UML modeling techniques
- Java programming skills
- GUI development with Swing

3. Data Management:

- File I/O operations
- Data serialization/deserialization
- CRUD operations implementation
- Search and filter algorithms

4. Professional Skills:

- Technical documentation writing
- User manual creation
- Code commenting and organization
- Testing and debugging

Challenges and Solutions

Challenge 1: Understanding Abstract Classes

- **Solution:** Researched OOP principles and implemented User as abstract class to enforce structure while allowing flexible implementations

Challenge 2: Multi-Level Inheritance

- **Solution:** Carefully designed hierarchy (User → Cashier → Manager) to reflect real-world relationships and maximize code reuse

Challenge 3: File-Based Persistence

- **Solution:** Implemented robust FileHandler with proper error handling and data validation

Challenge 4: GUI Design

- **Solution:** Studied Java Swing documentation and created user-friendly interfaces with consistent styling

Challenge 5: Role-Based Access

- **Solution:** Used polymorphism and instanceof checks to provide appropriate features based on user type

Future Enhancements

Potential improvements for future versions:

1. Database Integration:

- Replace file-based storage with MySQL/PostgreSQL
- Implement prepared statements for SQL injection prevention
- Add connection pooling for better performance

2. Enhanced Features:

- Sales tracking and reporting
- Inventory alerts for low stock
- Order management system
- Customer management
- Sales analytics and charts

3. Security Improvements:

- Password encryption (BCrypt/SHA-256)
- Session timeout
- Password strength requirements
- Audit logging

4. User Experience:

- More advanced search filters
- Export to Excel/PDF
- Print receipts
- Barcode scanning support

5. Technical Improvements:

- RESTful API for remote access
- Multi-user concurrent access
- Backup and restore functionality
- Cloud storage integration

Personal Reflection

This project provided valuable hands-on experience in applying Object-Oriented Programming concepts to solve real-world problems. The systematic approach of requirements gathering, UML modeling, implementation, and testing mirrors professional software development practices.

The most valuable learning experience was understanding how OOP principles like inheritance and polymorphism can significantly reduce code duplication and improve maintainability. The challenge of designing a proper class hierarchy taught the importance of planning before coding.

The project also reinforced the importance of clear documentation and user-centric design. Creating the user manual highlighted the need to think from the end-user's perspective and provide clear, step-by-step guidance.

Conclusion Statement

The Sweet Cupcake Shop Management System demonstrates a comprehensive understanding and practical application of Object-Oriented Programming principles. The system successfully meets all functional requirements while maintaining clean, well-documented, and maintainable code. The project serves as a solid foundation for understanding how OOP concepts translate into real-world software solutions.

8. REFERENCES

Books:

1. Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects* (11th ed.). Pearson Education.
2. Sierra, K., & Bates, B. (2005). *Head First Java* (2nd ed.). O'Reilly Media.
3. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.

Online Resources: 4. Oracle. (2024). *Java SE Documentation*. Retrieved from <https://docs.oracle.com/en/java/>

5. Oracle. (2024). *Java Swing Tutorial*. Retrieved from <https://docs.oracle.com/javase/tutorial/uiswing/>
6. Tutorials Point. (2024). *Object Oriented Programming Concepts*. Retrieved from https://www.tutorialspoint.com/java/java_object_classes.htm

UML Resources: 7. Object Management Group. (2024). *UML Specification*. Retrieved from <https://www.omg.org/spec/UML/>

8. Lucidchart. (2024). *UML Diagram Tutorial*. Retrieved from <https://www.lucidchart.com/pages/uml>

Course Materials: 9. Course Lecture Notes - CSE4006 Object Oriented Programming, Cardiff Metropolitan University (ICBT Campus), 2024-2025.

10. Assignment Brief - CSE4006 Object Oriented Programming, Cardiff Metropolitan University, January 2026

----END OF REPORT--